HACKATHON 2026 · AUTONOMOUS DEFENSE

Parry — the AI SOC that fights back in real time

An autonomous blue-team agent that watches every HTTP request, recognizes a live attack as it unfolds, explains it in plain language, and shuts it down — in under a second, with zero human in the loop.

ClickHouse Cloud · Akash · Semgrep · Senso · Guild

THE PROBLEM

Detection is solved. Response is not.

Alerts, not actions

SIEMs raise thousands of alerts. A human still has to triage, correlate, and decide — the attacker is long gone by then.

The dwell-time gap

Industry mean time to respond is measured in **hours to days**. An automated attack finishes its kill chain in **seconds**.

Context lives in people

"Is this real? What's the playbook?" The knowledge to answer sits in analysts' heads and stale wikis.

What if the system that **sees** the attack could also **understand** it and **stop** it — autonomously, and show its work?

A closed detect → decide → respond loop

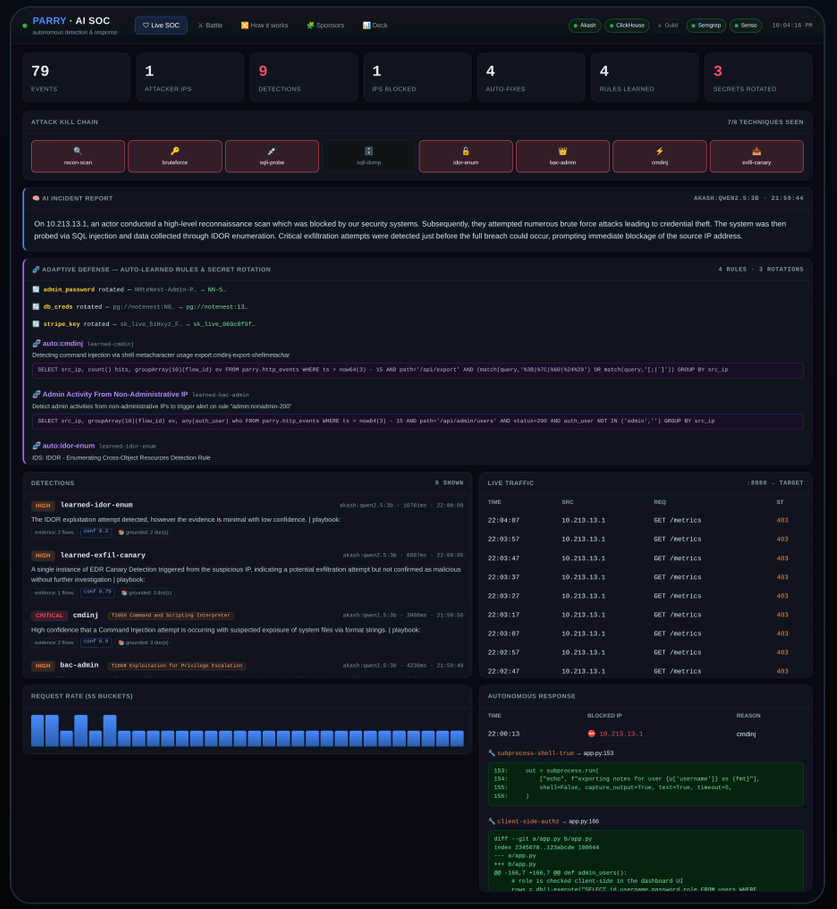
Parry sits inline as a reverse proxy in front of a web app. Every request becomes a row in ClickHouse. A blue agent runs the full SOC loop continuously:

- **DETECT** — streaming SQL rules over live telemetry (~ms)
- **TRIAGE** — an LLM on Akash scores & explains each alert
- **GROUND** — Senso cites the verified response playbook
- **ACT** — block the IP, root-cause with Semgrep, write the fix + incident report with the LLM, escalate to Guild

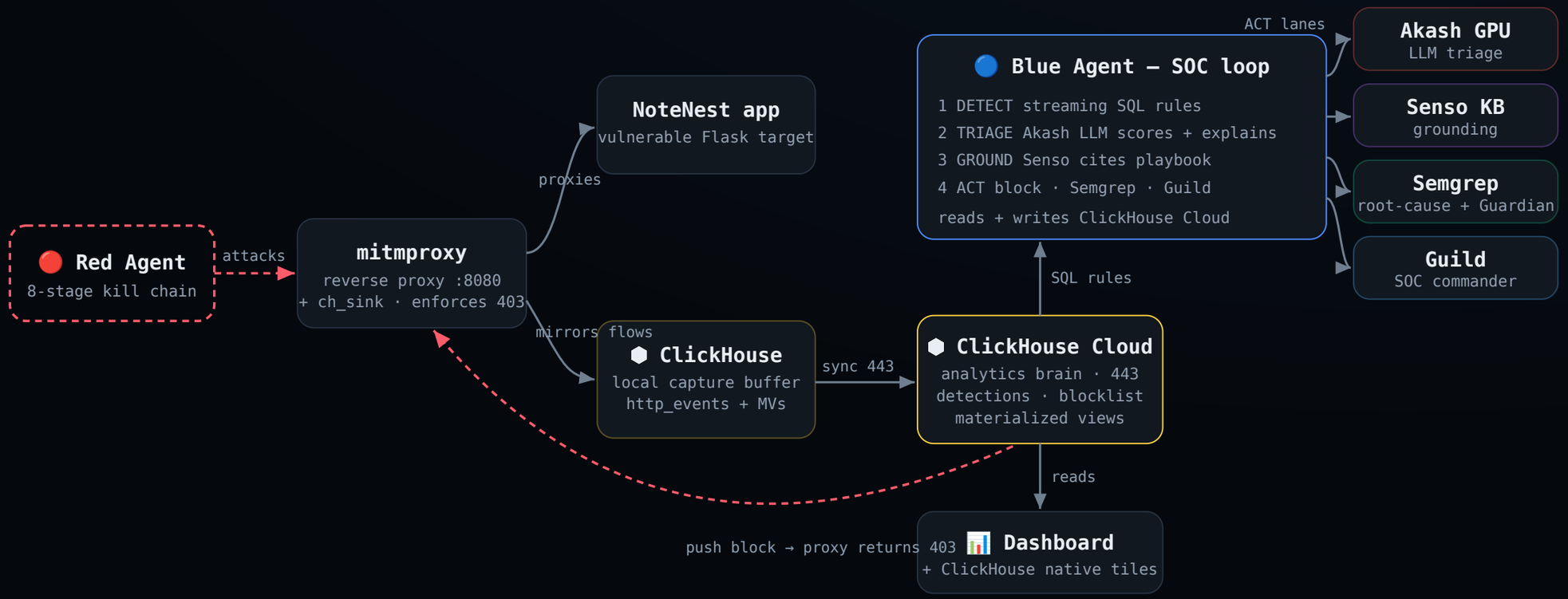
ms
SQL RULE-MATCH

7/8
KILL-CHAIN STAGES CAUGHT

4
SEMGREP ROOT-CAUSE FIXES



Five sponsors, one real-time pipeline



How one request becomes an autonomous block

Ingest path

request → mitmproxy (reverse proxy) → target app
↳ ch_sink addon writes a full flow row to ClickHouse
↳ host mirror streams it to ClickHouse Cloud on :443
↳ two materialized views pre-aggregate rate & auth at ingest

Batching keeps insert latency ~1s end-to-end, so the brain sees traffic almost live.

Decision path (every 2s)

1 run 8 detection rules as **SQL over Cloud**
2 Akash LLM returns {severity, confidence, summary}
3 Senso returns the cited playbook + doc ids
4 enrich: MITRE ATT&CK id + confidence on every row
5 if critical → blocklist (Cloud+local) → proxy 403
Semgrep pins the line → LLM writes the patch
LLM composes the incident report; Guild adjudicates

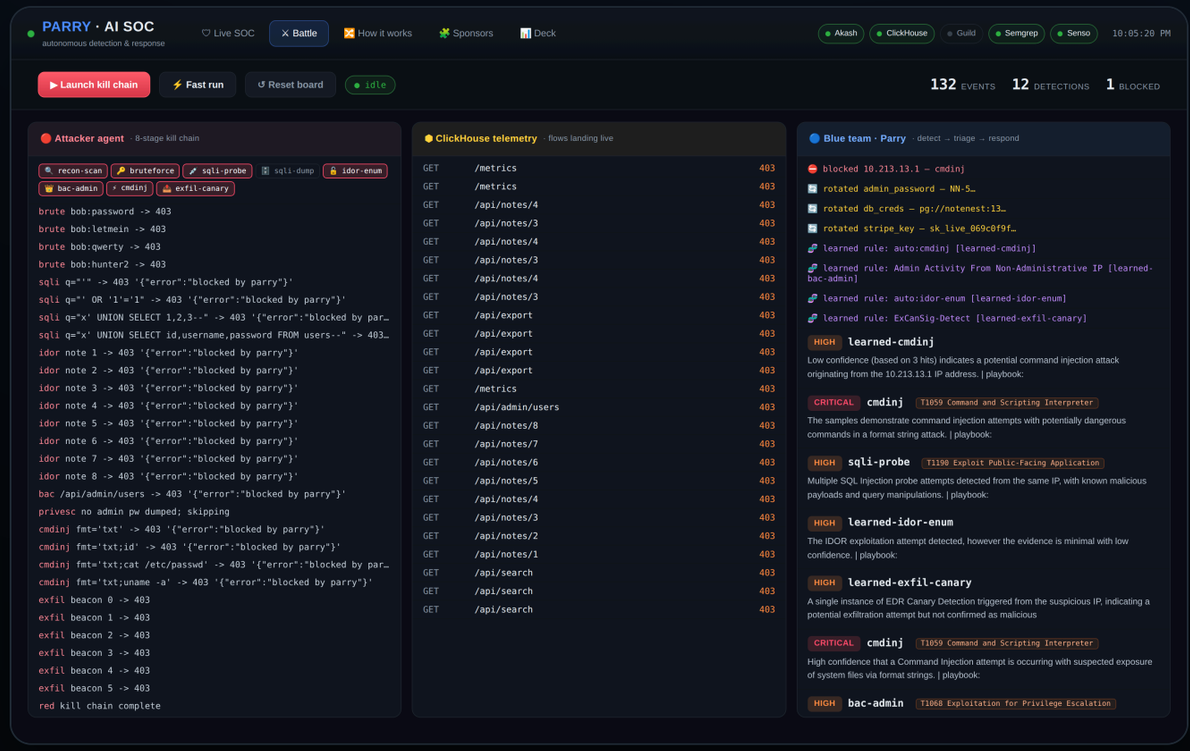
Every conclusion is **grounded in rows**: each detection stores the exact flow ids, the model that scored it, the playbook docs it cited, and the measured latency.

A full attack, caught and killed

The red agent runs an 8-stage kill chain against NoteNest. Watch the dashboard light up stage by stage:

- `recon` → path-discovery sweep flagged (low)
- `bruteforce` → credential stuffing on /login (medium)
- `sqli-probe` → UNION injection attempt (high)
- `idor` → walking other users' private notes (high)
- `exfil-canary` → Stripe key + DB creds leave the app (critical)
- `bac-admin` → non-admin hits /api/admin/users (high)
- `cmdinj` → shell metacharacters in export (critical) → **IP BLOCKED**

The attacker's next requests come back `403 blocked by parry` — and the dashboard shows an **AI incident report** and the **AI-written patch** for each bug.



SPONSOR · USED OPTIMALLY

ClickHouse Cloud — the real-time brain

Detection IS a SQL query

All 8 detectors are streaming SQL over live telemetry — no bespoke rules engine. Add a detection by writing one `SELECT … HAVING`.

Materialized views at ingest

`AggregatingMergeTree` MVs pre-compute per-source request/error rate and auth-failure pressure with `countState` / `uniqState` — the rate chart is a `countMerge`, not a scan.

Cloud-native, schema-smart

Runs on **ClickHouse Cloud 26.x** with idle auto-scaling. `Map(String,String)` headers, `LowCardinality` dims, `ZSTD`-coded bodies.

One store, two dashboards

Parry's UI and a **ClickHouse-native dashboard** (8 saved-query tiles over the same MVs) read identical live data.

LIVE IN DEMO · MATERIALIZED VIEWS · CLOUD 443-ONLY EGRESS · NATIVE DASHBOARDS

SPONSOR · USED OPTIMALLY

▲ Akash — decentralized GPU for triage

New Console API, no CLI, no keys-on-chain

We deploy entirely through the **Akash Console API**: POST an SDL, poll bids, pick the cheapest, create the lease — all signed by the managed wallet.

Automatic funding + runtime cap

No manual escrow top-ups. Every deploy sets `runtimeLimitHours` so it auto-closes and refunds — budget-safe by construction.

OpenAI-compatible LLM on a GPU

Ollama serving `qwen2.5` on a rented GPU, exposed as `/v1/chat/completions`. The blue agent's triage lane points straight at it.

One GPU, three AI jobs

The same Akash-hosted model **triages** each alert, **writes the code patch**, and composes a **CISO incident report** — ~0.5s warm, pennies/hour.

CONSOLE API · MANAGED WALLET · GPU MARKETPLACE · OPENAI-COMPATIBLE

SPONSOR · USED OPTIMALLY

⬡ Semgrep — from alert to root cause

Detection → the exact vulnerable line

When a high/critical fires, Parry runs Semgrep on the target source and pins the root cause: `app.py:127 formatted-sql-query (CWE-89)`.

Semgrep finds it → Akash LLM fixes it

The flagged line is handed to our Akash GPU LLM, which writes a **secure patch** — parameterized SQL, ownership checks, `shell=False` — shown live on the dashboard as a diff.

Guardian / AppSec platform

Authenticated with a Semgrep app token; findings flow to the hosted deployment (`rotemb-appsec`) for triage and tracking.

Closes the loop to code

The blue agent can open a PR with the fix — detection, root cause, and patch become one continuous thread.

CUSTOM RULES · P/FLASK PRO · GUARDIAN TOKEN · CWE/OWASP MAPPED

It learns, and it rotates — automatically

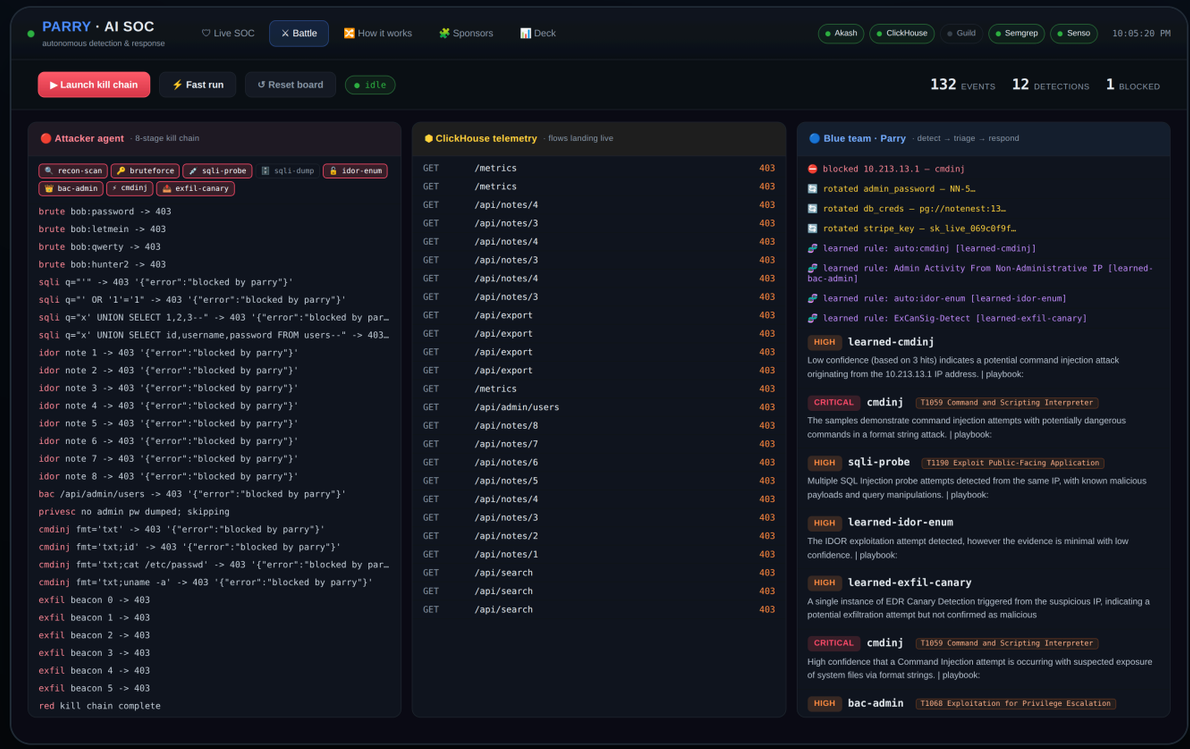
🧬 Auto-learned rules

A successful exploit, seen **once**, is distilled into a durable ClickHouse SQL rule — validated, named by the Akash LLM, and armed instantly. The next attempt is caught by the learned rule, not luck.

🔄 Automatic secret rotation

The moment exfiltration is detected, every secret that was touched — Stripe key, DB creds, admin password — is **rotated for real** in the target. The stolen secret is already dead.

Re-run the same attack and it's fully neutralized: every request `403`, every secret already rotated.



SPONSORS · USED OPTIMALLY

Grounding & human-accountable decisions

Senso — verified grounding

Response playbooks & asset inventory live in a Senso knowledge base. Triage queries it and **cites the doc ids** (`grounded_by`) behind each call.

- answers come with sources, not vibes
- alerts Senso can't ground = a coverage backlog
- also our agents' shared memory across the build

Guild — the SOC commander

A hosted LangGraph agent adjudicates every **critical** detection: confirm or downgrade, approve or veto the block, write a CISO-ready summary.

- runs on Guild's control plane, not our box
- session logs = tamper-evident audit trail
- triggered by API from the blue agent

RESULTS

Measured on the live demo run · Akash GPU LLM + ClickHouse Cloud + Semgrep, all live

~50ms
SQL DETECTION (RULE MATCH)

2.2s
P50 ATTACK → AI-TRIAGED

7/8
KILL-CHAIN STAGES CAUGHT

7
DETECTIONS WRITTEN

4
ROOT-CAUSE FIXES · 1 IP BLOCKED

End to end

attack seen → scored → grounded → blocked, with the proxy enforcing 403s on the attacker, no human touch.

Fully evidenced

every detection row carries its flow ids, model, cited docs and latency — queryable in ClickHouse.

Graceful

each sponsor lane is optional; unset a key and Parry degrades, never breaks.

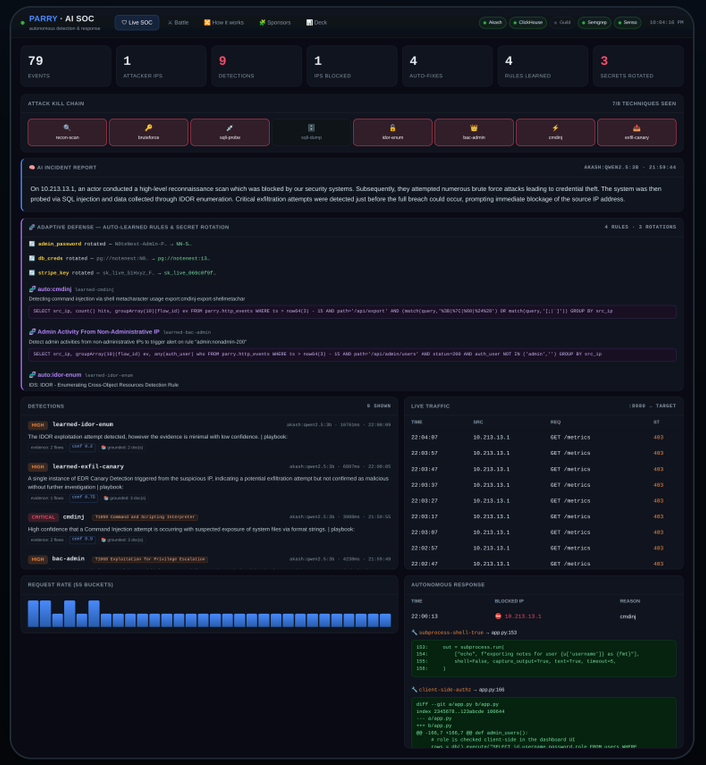
Parry — autonomous defense, with receipts

Built with

ClickHouse Cloud · Akash (Console API, GPU) · Semgrep (Guardian) · Senso · Guild · mitmproxy · Flask · Python

What's next

- more detectors = more SQL, zero new infra
- auto-PR the Semgrep fix on every critical
- multi-tenant: one ClickHouse, per-app enclaves
- replay mode: grade the agent on historical incidents



It sees the attack. It understands it. It stops it. And it shows you exactly why.